

Serving Plan: Real-Time Adaptive Music Recommendation

Internal Team Review Draft

Team: **Spoty Boys** — Role Focus: **Serving (Vishal)**

Date: March 3, 2026

1. Scope of This Document (Serving Only) This document specifies the **operational requirements** and the **system integration** for serving the ML recommendation feature in our real-time adaptive music recommender integrated with Navidrome. It is written to be shareable and reviewable by the full team, and to directly satisfy the proposal requirements:

- Estimate operational requirements for serving (RPS, latency, timeouts, scaling plan) with justification.
- Describe how model outputs translate into an outcome in the real system (what changes in playback/queue).

2. Key Assumptions (Explicit and Reviewable) We will design for a **class-project deployment scale** while still following production-grade patterns (fallbacks, observability, caching, versioning). Assumptions below are intentionally conservative to avoid over-engineering while staying realistic:

- **Active listeners (target test):** 100 concurrent users during load tests (smaller during normal demo).
- **Average track length:** ≈ 3 minutes (typical streaming listening).
- **Primary triggers:** recommendation refresh on song **end** or **skip**.
- **Recommendation objective:** maintain a short *Up Next* queue to keep playback smooth.

Why these assumptions? They map directly to measurable events (end/skip), and let us compute RPS and latency targets from first principles.

3. System Flow (Bird's-Eye Diagram)

Why this architecture?

- REST is sufficient at our expected throughput (~ 0.5 – 1 RPS steady), keeping complexity low.
- Redis supports fast session reads/writes (enables adaptivity without heavy DB load).
- Postgres provides durable event history for periodic retraining and offline evaluation.

4. Serving Decisions Summary (One Table for Quick Team Review)

5. Inference Call Pattern (When Do We Call the Model?) **Decision:** Refresh on end/skip + keep queue of 5.

- **Trigger events (primary):** Song **end** and **Skip**.
- **Action:** On trigger, call recommender and **refill/refresh** the **Up Next queue** to length 5.

Justification: end/skip are the only moments that affect the next track immediately; keeping a queue of 5 provides a buffer so even small inference hiccups do not interrupt playback.

6. Requests Per Second (RPS) Estimate **Given:** 100 active users. If average track length is ≈ 3 minutes, per-user triggers per minute $\approx 1/3 = 0.33$. Total RPS:

$$\text{RPS} \approx \frac{100 \times 0.33}{60} \approx 0.48$$

Targets: 0.5–1 RPS steady, 5 RPS burst.

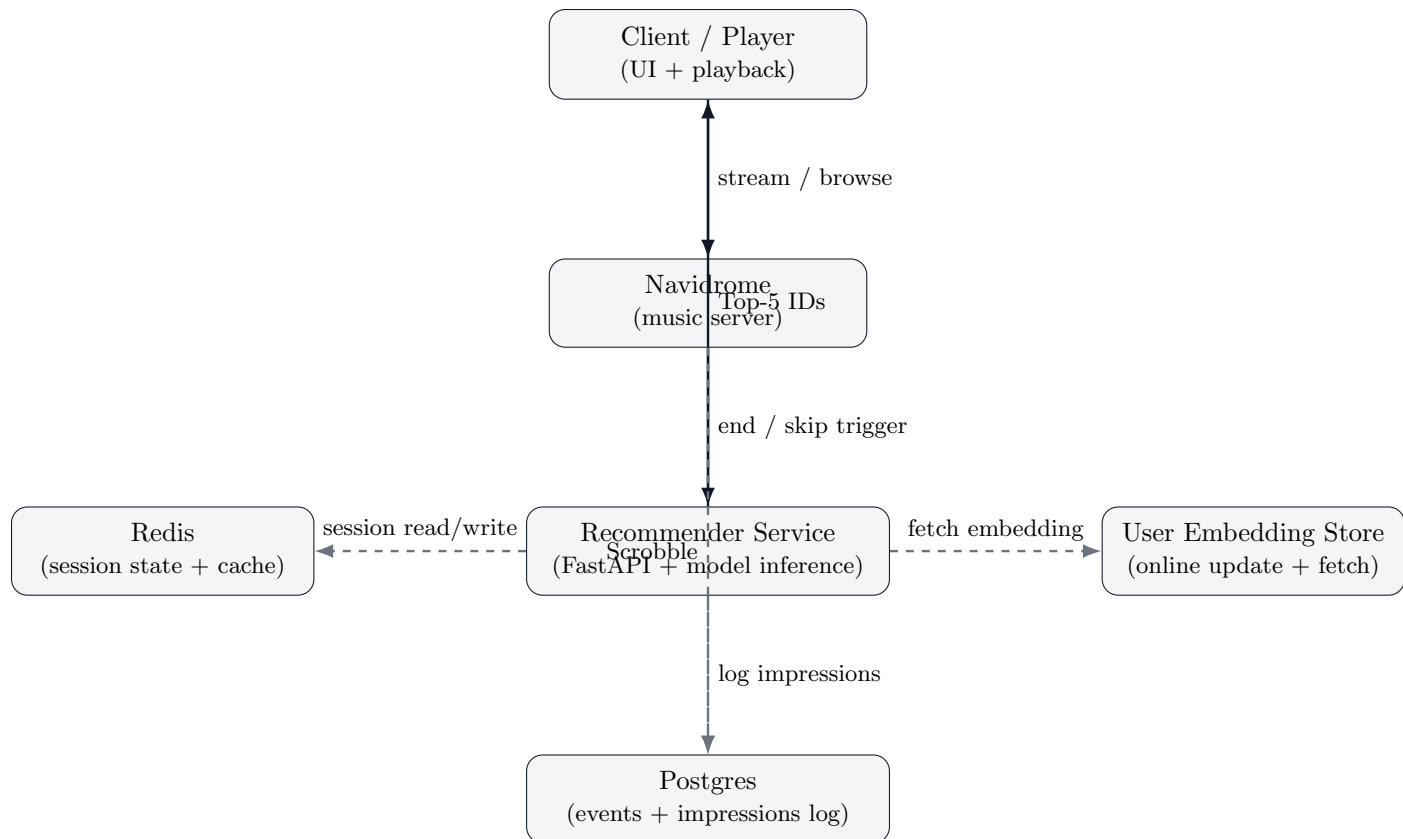


Figure 1: Online serving + logging flow. User playback triggers inference. Events and impressions are logged for retraining.

7. Latency SLOs and Fallbacks Targets (customized):

- **p50 latency** < 100ms
- **p95 latency** < 300ms
- **Hard timeout:** 800–1000ms $\Rightarrow$ fallback to baseline

Why these thresholds? With a buffer of 5 songs, median sub-100ms feels instant; p95 300ms stays responsive under contention; timeout prevents tail latency from harming playback.

8. Model Output $\rightarrow$ Real System Outcome Outcome: update *Up Next* queue (length 5) with model-ranked song IDs.

9. API Contract (Serving Interface) We expose a minimal REST inference API (sufficient for our RPS):

- **Request:** `user_id`, `recent_song_ids` (N=5), `event_type` (END/SKIP), optional context (timestamp, device).
- **Response:** `recommended_song_ids` (Top-5), optional scores, `model_version`.

Why REST? At ~ 0.5 –1 RPS steady, REST is simplest and reliable; streaming adds operational overhead that is unnecessary for the MVP.

10. Two-Stage Serving (Candidates then Rank) To keep CPU inference predictable and fast:

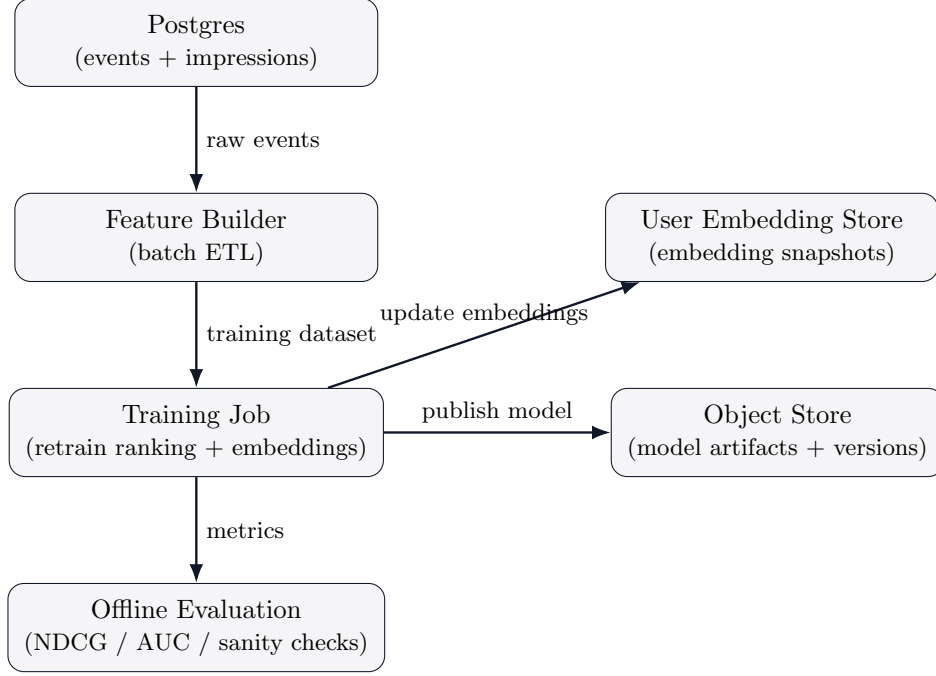


Figure 2: Offline retraining loop. Logged events are transformed into training data, models are retrained and evaluated, and new artifacts are published for serving.

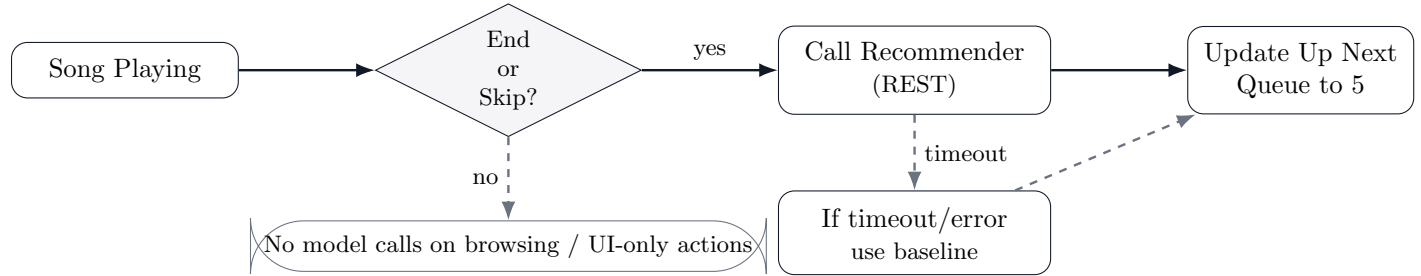


Figure 3: Decision flow: only end/skip triggers call the recommender. This limits load, keeps behavior predictable, and matches the moments when the next-track choice matters.

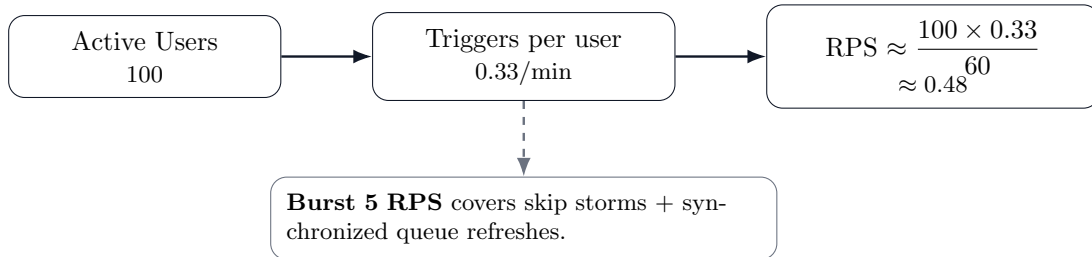


Figure 4: RPS derives from user concurrency and the cadence of end/skip events, giving a grounded throughput estimate.

- **Candidate generation (cheap):** popularity / random-from-liked / content similarity $\rightarrow C$ candidates (suggest $C = 100$).
- **Ranking (ML):** score candidates and return Top-5.

Table 1: Serving Requirements and Integration Decisions (with justification)

Category	Decision / Target	Justification (Why we do it)
Inference call pattern	Refresh on end/skip + keep queue of 5	End/skip are the moments when the next recommendation matters; a queue buffer prevents playback gaps and avoids calling the model on every UI interaction.
Throughput (RPS)	100 active users $\Rightarrow$ RPS $\approx 100 \times 0.33/60 \approx 0.48$ Steady: 0.5–1 RPS Burst: 5 RPS	Derived from track duration (~ 1 trigger/3 min). Burst covers skip storms + synchronized refreshes; ensures we size for realistic peaks not just averages.
Latency SLO	p50 < 100ms p95 < 300ms Timeout: 800–1000ms Fallback on timeout	Queue refill must feel instant. Hard timeout prevents stalled playback; fallback ensures continuity when the model is slow/unavailable.
Output contract	Return Top-5 ranked song IDs (+ optional scores)	Song IDs are the native system action unit (play/queue). Scores are optional for debugging and evaluation.
Candidate strategy	Two-stage: 100 candidates $\rightarrow$ rank $\rightarrow$ Top-5	Avoid scoring the entire library every request. Keeps CPU inference fast and makes model iteration easy.
Caching	Per-user cached Top-5 for 30–120s ; session-state in Redis	Caching reduces repeated scoring during bursts and helps meet p95 latency. Short TTL preserves adaptivity.
Fallbacks	Popularity / random-from-liked / content-similarity	Guarantees playback never breaks; baselines are easy to debug and provide an anchor for measuring ML lift.
Storage	Postgres for events; Redis for session; object storage for models	Events must be durable for retraining; session must be fast/ephemeral; model artifacts must be versioned for reproducibility/rollback.
Observability	Latency, RPS, errors, fallback rate, cache hit rate; plus skip/listen-through	We need both system health and product impact to validate choices and debug regressions.

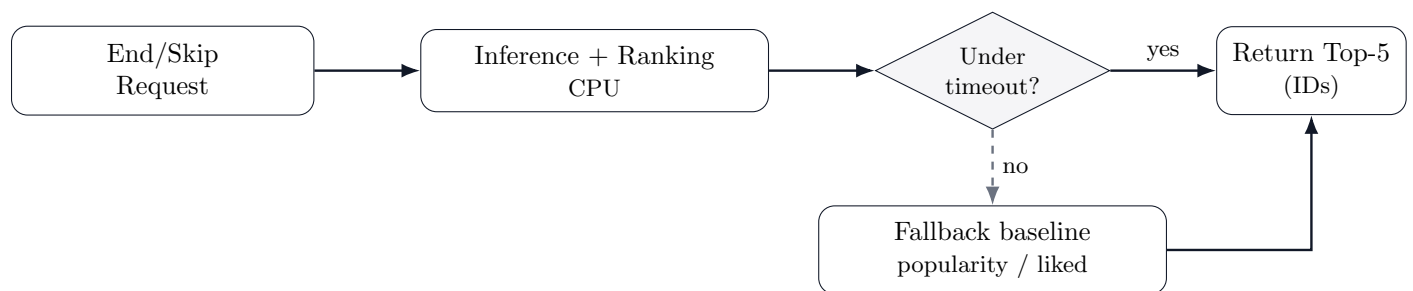


Figure 5: Latency policy: strict timeout bounds worst-case UX. If ML is slow, we return safe recommendations so playback never breaks.

Why two-stage? It avoids scoring the entire catalog on every request and makes model iteration easy.

11. Caching + State

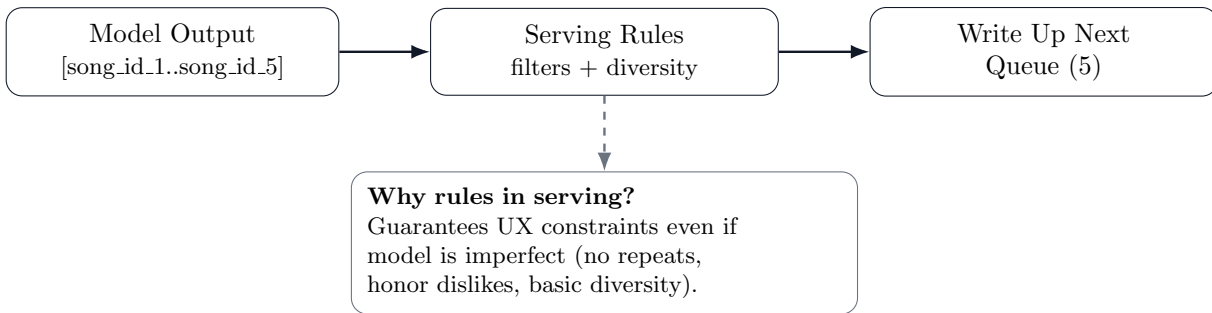


Figure 6: Output-to-action mapping: the recommender returns IDs, serving applies safety/business rules, then updates the queue.

- Cache per-user Top-5 for **30–120s** to smooth bursts and reduce repeated scoring.
- Store session features (last-N songs, skip counters, last refresh time) in **Redis**.

Why? Cache improves p95 tail; Redis supports low-latency session reads/writes needed for adaptivity.

12. Fallback Strategy (Never Break Playback) If inference errors or exceeds 800–1000ms:

- Popularity-based (global / recent)
- Random-from-liked (if available)
- Content-similarity (if metadata exists)

Why? Guarantees continuity and provides a baseline for measuring ML improvement.

13. Storage Dependencies (Serving View)

Table 2: Serving Data Dependencies

Data Type	Suggested Store	Why
Playback/feedback events	Postgres	Durable ground truth for retraining and offline evaluation; supports popularity baselines.
Session state (last- N , counters)	Redis	Fast and ephemeral; ideal for real-time adaptation features.
Model artifacts + versions	Object store (S3/MinIO)	Versioning enables reproducibility, rollbacks, and consistent evaluation.

14. Observability (What We Measure) System metrics

- RPS, p50/p95 latency, timeout rate, error rate
- Cache hit rate, fallback rate

Product metrics

- Skip rate (overall + early skip), listen-through rate, like/dislike rate
- Diversity indicators (repeat rate, unique artists in recent window)

Why? We must validate SLOs and verify the recommender improves user outcomes.

15. Safety / Business Rules (Serving-Side) Serving enforces constraints on model output:

- No repeats among last M songs (suggest $M = 20$)
- Hard filter explicit dislikes
- Limit consecutive same-artist tracks (e.g., max 2)
- Optional exploration knob ϵ (e.g., 10% novelty)

16. Deployment Plan (Chameleon-Friendly)

- FastAPI recommender (CPU inference), Redis (session/cache), Postgres (events)
- Scale horizontally if needed; protect p95 with caching and debouncing (max 1 call/sec/user)

17. Open Questions for Team Review

- Can we capture end/skip/play duration events from Navidrome reliably?
- Where do we implement “Smart Queue” writeback (client plugin vs middleware)?
- Stable song IDs and metadata availability for content baselines?

Owner: Vishal (Serving). Add comments directly for team alignment.

18. Feasibility Review: User Data, Cold Start, and Model Design

This section summarizes our current doubts regarding (1) availability of user interaction data and (2) model architecture strategy. The goal is to clarify what is realistic, what risks exist, and what architectural decisions mitigate those risks.

18.1 Question 1: Can We Reliably Obtain User Profile + Actions?

Core Concern: Our entire personalization flow depends on capturing user behavior signals (play, skip, duration, like/dislike). If we cannot reliably capture these signals, long-term user modeling will be weak and early recommendations may perform poorly.

Reality Assessment

There are three realistic implementation paths:

1. Navidrome Plugin-Based Event Capture

- Use Navidrome’s plugin system to listen for playback-related events.
- Write events (play, skip proxy, duration, feedback) into our own Postgres table.
- Pros:
 - Full control over event schema
 - Clean training dataset
 - Production-aligned design
- Cons:
 - Requires deeper integration effort
 - Must validate what exact events are exposed

2. External Scrobbling (Last.fm / ListenBrainz)

- Forward listening events to external service.
- Use exported listening history as training signal.
- Pros:

- Minimal implementation effort
- Ready-made historical logging
- Cons:
 - Less control over event semantics
 - Harder to extend with custom feedback fields

3. Client-Side Event Logging

- Intercept end/skip/play events in the player layer.
- Send structured events to our recommender backend.
- Pros:
 - Most flexible
 - Explicit skip detection possible
- Cons:
 - Requires UI/client modification

Cold Start Risk Analysis

If we rely purely on collaborative filtering:

- First few users $\Rightarrow$ extremely sparse data
- Predictions may be unstable
- System may appear random

However, if we include:

- Popularity baseline
- Content-based similarity
- Session-based similarity

then cold start becomes manageable.

Conclusion: The flow is realistic **if and only if** we explicitly build our own structured event log from day one. We should not assume Navidrome internally provides a full training-ready dataset.

18.2 Question 2: Two-Model Strategy (Retrieval + Ranking)

Proposed Idea:

- Model A: Embedding-based similarity search (guarantee songs exist)
- Model B: Full recommender using user profile + actions

Interpretation in Recommender System Terms

This corresponds to the industry-standard:

Candidate Generation (Retrieval) $\rightarrow$ Ranking Model

Model A: Session / Similarity-Based Retrieval

- Input: last 1–5 songs
- Output: 50–100 nearest-neighbor candidate songs
- Implementation:

- Content embedding similarity
- Small neural encoder
- Metadata similarity

Strengths:

- Works with zero user history
- Strong short-term mood continuity
- Guarantees candidate availability

Weaknesses:

- Can get stuck in narrow genre loops
- Does not capture long-term taste

Model B: Personalized Ranking Model

- Input:
 - User embedding (long-term)
 - Session embedding (short-term)
 - Candidate song features

- Output: ranking score for each candidate

Strengths:

- Incorporates user history
- Balances long-term and short-term intent
- Improves with more data

Cold Start Behavior:

- If user history is small $\Rightarrow$ rely more heavily on Model A
- As interactions grow $\Rightarrow$ increase weight of user embedding

18.3 Recommended Architecture Decision

Rather than two competing recommenders, we propose:

1. Stage 1: Candidate Generation

- Use embedding similarity over recent songs
- Add small percentage of exploratory/popular tracks

2. Stage 2: Ranking

- Score candidates using:

$$\text{Score}(\text{song}) = f(\text{User Embedding}, \text{Session Embedding}, \text{Song Features})$$

- Return Top-5

3. Fallback

- If ranking unavailable $\Rightarrow$ return top candidates from Stage 1

Benefits of This Design:

- Guaranteed coverage (always songs available)
- Cold-start robustness
- Scales naturally as more user data accumulates
- Aligns with modern large-scale recommender architecture

18.4 Final Summary for Team Alignment

- The system is realistic if we deliberately build a structured event pipeline.
- Cold start is solvable using popularity + session similarity.
- Two-model strategy is not redundant — it is actually best practice (retrieval + ranking).
- Personalization quality will improve over time as user interaction data grows.

Decision Required from Team:

- Which event capture method will we commit to?
- Do we formally adopt the two-stage retrieval + ranking architecture?

19. Retraining Strategy: What User Information Can We Collect?

This section summarizes what user-level information we can realistically collect and how each signal contributes to model retraining. The goal is to design a robust feedback loop that improves personalization over time.

19.1 Types of User Signals for Retraining

We divide signals into three categories:

A. Implicit Feedback (High Volume, Low Friction)

- **Play / Start Event**
 - User begins listening to track i
 - Weak positive signal
- **Listen Duration / Completion**
 - Percentage of track consumed
 - Stronger signal than play
 - Completion ($>80\%$) $\Rightarrow$ strong positive
- **Skip**
 - Early skip ($<20\text{--}30$ seconds) $\Rightarrow$ strong negative
 - Also signals session/mood change
- **Replay**
 - Track played again shortly after
 - Very strong positive preference
- **Search + Manual Selection**
 - Explicit user intent
 - Strong signal for long-term taste

- **Queue Modifications**

- Removing a recommended track $\Rightarrow$ negative
- Adding a track $\Rightarrow$ positive

Why implicit signals matter: They provide continuous training data without interrupting the user experience.

B. Explicit Feedback (Low Volume, High Precision)

- Like / Dislike (Green / Red)
- Optional rating (1–5)
- “More like this” / “Not in mood”

These signals provide high-quality labels for ranking model supervision.

C. Contextual Features

- Time of day / day of week
- Device type
- Session start seed track

Context allows the model to capture short-term listening patterns (“mood”) beyond static user taste.

19.2 Minimum Data Required for Effective Retraining

At minimum, we must log:

- User ID
- Session ID
- Track ID
- Timestamp
- Event type (PLAY, SKIP, LIKE, DISLIKE, etc.)
- Listen duration (if available)
- Recommendation source (RECOMMENDED vs MANUAL)
- Model version that generated recommendation

Additionally, we should log recommendation impressions:

- List of candidate tracks shown
- Ranked Top-K returned
- Which track was selected

Why impression logging is critical: Without logging what was shown, we cannot properly train ranking models or evaluate bias.

19.3 How Signals Improve Each Model Component

A. Long-Term User Model

Uses:

- Likes
- Replays
- High completion tracks
- Repeated listening patterns

Output:

$$\text{User Embedding} = g(\text{historical interactions})$$

B. Short-Term Session Model

Uses:

- Last N songs
- Recent skips
- Rapid genre shifts

Output:

$$\text{Session Embedding} = h(\text{recent activity})$$

C. Ranking Model

Training labels derived from:

- Positive: Like, Replay, Completion > 80%
- Negative: Early Skip (< 20–30s), Dislike
- Neutral: Partial listen

Optimization objective:

$$\text{Score}(\text{song}) = f(\text{User Embedding}, \text{Session Embedding}, \text{Song Features})$$

19.4 Cold Start Considerations

If user history is sparse:

- Rely more heavily on session similarity (retrieval model)
- Blend in popularity-based candidates
- Gradually increase weight of user embedding as interactions grow

This ensures acceptable recommendations even in early phases.

19.5 Key Design Principle

Playback must generate structured training data from day one.

Therefore:

- We must explicitly implement event logging.
- We cannot assume the base music server provides a training-ready dataset.
- Retraining quality is directly proportional to the quality of event capture.

Team Decision Required:

- What exact events can we reliably capture?
- Where will event logging be implemented (plugin, client-side, middleware)?

20. Data Plan (Signals, Sources, and Logging)

Goal Our recommender requires **structured interaction data** to train/refresh user embeddings and to supervise the ranking model. The key signals we need include **skip/stop/replay**, **listen duration (completion)**, and **recommendation impressions** (what was shown vs what was chosen).

Reality Check: What Navidrome/Plugins Provide We reviewed Navidrome’s internal plugin ecosystem and external scrobbling integrations. Internal plugins primarily extend functionality and do **not** provide a complete, training-ready interaction dataset (e.g., explicit skip events, precise durations, impression logs). External scrobbling (e.g., Last.fm) provides **scrobbles** (completed/qualified listens) and a **now-playing** state, but does not reliably expose explicit button-level events (skip/stop) or precise listen duration for all tracks. Therefore, external scrobbling is treated as an **auxiliary** signal rather than our primary training source.

Primary Plan: First-Party Interaction Logging (Training-Ready) To ensure retrainability, we will implement a **first-party event logging layer** that produces a consistent schema from day one. At minimum, we log:

- **Identity + context:** `user_id`, `session_id`, `track_id`, `timestamp`, `device/context` (optional)
- **Interaction events:** `PLAY`, `END`, `SKIP`, `PAUSE/STOP` (if available), `REPLAY` (derived)
- **Consumption:** `listen_duration` and/or `completion_ratio` (when measurable)
- **Attribution:** `source` (`RECOMMENDED` vs `MANUAL`), `model_version`
- **Impressions (required for ranking):** candidate list returned, Top- K shown, selected track

Design principle: playback must generate structured training data from day one; we do not assume the music server provides a full training dataset.

Labeling: Turning Logs into Training Signals We convert raw events into supervision for ranking and embedding updates:

- **Positive:** completion > 80%, replay within a short window, explicit like (if implemented)
- **Negative:** early skip (e.g., <20–30% completion or <20–30s), explicit dislike (if implemented)
- **Neutral/weak:** partial listens without skip, passive browsing

These labels support (i) **online user embedding updates** and (ii) periodic **offline retraining** with reproducible datasets.

Cold Start: Offline Pretraining with MTG-Jamendo To avoid cold-start failure before interaction logs accumulate, we will use **MTG-Jamendo** for offline training:

- Train content embeddings from **audio + tags/metadata**
- Use embedding similarity for candidate generation (retrieval stage)

This enables reasonable recommendations immediately, then improves as first-party interaction data grows.

Optional Auxiliary Data: Last.fm (If Available) If enabled, Last.fm provides additional historical signals:

- Scrobble history (timestamped listens)
- Now-playing state (usable for coarse session boundaries)

However, due to limited event granularity, it is used for **bootstrap and sanity checks**, not as the primary supervision source.

Open Implementation Choice (Team Decision) We will commit to **one** concrete capture path for first-party logging:

- Navidrome-side integration (plugin / server hook), or
- Client-side logging (player UI events), or
- Middleware layer (intercept/augment playback actions)

Selection criterion: ability to reliably capture **end/skip** and **consumption duration**, plus impression attribution.